Rodrigo Fernandez

Senior Node.js / Python Engineer
Katy, Texas | +1 (407) 801-1287 | www.linkedin.com/in/rodrigobermejo-fernández-9a5a0664 | rodrigobfdev@gmail.com

Summary

Senior Node.js / Python Engineer with deep experience building cloud-native SaaS, FinTech, healthcare, eCommerce, and enterprise workflow platforms using **Node.js 14.x–20.x**, **Python 3.8–3.12**, **FastAPI 0.95.x–0.110.x**, **Django 3.2.x–5.0.x**, **Express 4.x**, **NestJS 7.x–10.x**, PostgreSQL, MongoDB, Redis, AWS, Docker, Kubernetes, and CI/CD. Specialized in scalable APIs, async workers, event-driven integrations, payment reliability, data pipelines, authentication, observability, and legacy modernization, with strong ownership of production bugs, migrations, performance tuning, and secure backend architecture across global distributed product teams.

Skills

- **Backend: Node.js 14.x–20.x**, **Python 3.8–3.12**, **TypeScript 4.x–5.x**, JavaScript ES6+, **Express 4.x**, **NestJS 7.x–10.x**, **FastAPI 0.95.x–0.110.x**, **Django 3.2.x–5.0.x**, Flask 1.x–2.x, REST APIs, GraphQL, WebSockets, Microservices, Serverless APIs
- **Databases: PostgreSQL 12.x–16.x**, MySQL 8.x, MongoDB 4.x–7.x, Redis 5.x–7.x, DynamoDB, Elasticsearch 7.x–8.x, SQLAlchemy, Prisma, TypeORM, Sequelize, Django ORM
- **Cloud & DevOps: AWS**, Lambda, ECS, EKS, S3, SQS, SNS, API Gateway, CloudWatch, IAM, Docker, Kubernetes, Terraform, GitHub Actions, GitLab CI/CD, Jenkins
- **Testing & Quality:** Jest, Mocha, Pytest, Unittest, Supertest, Postman, Swagger/OpenAPI, Contract Testing, Integration Testing, Load Testing, SonarQube
- **Architecture:** Event-Driven Systems, Queue-Based Workers, API Gateway Design, Domain-Driven Design, Authentication, OAuth2, JWT, RBAC, Observability, Distributed Tracing

Experience

Lightedge — Senior Software Engineer *(Apr 2020 – Mar 2026)*

- Built secure SaaS and FinTech workflow platforms using **Node.js 14.x–20.x**, **Python 3.8–3.12**, **FastAPI 0.95.x–0.110.x**, **NestJS 7.x–10.x**, PostgreSQL, Redis, AWS, Docker, and Kubernetes for high-volume enterprise clients.
- Designed **REST API** and event-driven backend services for billing, customer onboarding, document processing, and reporting workflows, improving service reliability by **38%** through cleaner module boundaries and async job isolation.
- Implemented new payment reconciliation features with **Node.js**, Python workers, SQS queues, PostgreSQL transaction tables, and idempotency keys, reducing duplicate financial records by **47%** during retry-heavy provider callbacks.
- Migrated legacy Express services from older JavaScript patterns into **TypeScript 4.x–5.x** with NestJS modules, DTO validation, dependency injection, and OpenAPI contracts while keeping production APIs backward compatible.
- Resolved a critical production bug where Python background jobs silently skipped failed webhook payloads by adding structured retry logic, dead-letter queues, correlation IDs, and CloudWatch alarms.

- Optimized PostgreSQL-heavy reporting APIs by rewriting slow joins, adding composite indexes, batching ORM calls, and caching summary data in Redis, cutting average response time by **52%**.
- Built secure authentication and authorization flows using **OAuth2**, JWT, refresh tokens, RBAC policies, and tenant-aware middleware to protect sensitive healthcare and financial data across multi-tenant environments.
- Improved CI/CD pipelines with GitHub Actions, Docker image scanning, Pytest, Jest, migration checks, and staged AWS deployments, reducing failed releases by **34%** across backend repositories.
- Developed Python data ingestion services with **FastAPI**, Pandas, SQLAlchemy, S3, and scheduled workers to normalize customer activity data for dashboards, billing exports, and operational analytics.
- Migrated multiple services from Python 3.8 to **Python 3.11–3.12** by fixing dependency conflicts, async compatibility issues, Docker base images, and test failures before production rollout.
- Added observability with structured JSON logs, Prometheus metrics, OpenTelemetry traces, and actionable alert rules, helping teams identify timeout, memory, and database pool issues faster.
- Mentored backend engineers on **Node.js**, Python, testing strategy, code review quality, and production debugging while documenting reusable patterns for APIs, workers, migrations, and cloud deployments.

NIX United —Software Engineer *(Oct 2016 – Mar 2020)*

- Developed healthcare, logistics, and eCommerce backend systems using **Node.js 8.x–12.x**, **Python 3.6–3.8**, **Django 1.11.x–3.0.x**, Flask 1.x, Express 4.x, PostgreSQL, MySQL, Redis, Docker, and AWS.
- Created RESTful backend modules for patient scheduling, order tracking, inventory updates, and customer notifications, improving operational workflow completion speed by **31%** for client-facing platforms.
- Fixed a recurring race condition in Node.js order processing by adding database-level locks, Redis-based queue sequencing, and idempotent status transitions across Express service handlers.
- Migrated legacy Python 2.7 utility scripts into **Python 3.6–3.8** services by replacing deprecated libraries, updating encoding logic, rewriting tests, and validating production batch outputs.
- Built Django admin extensions, Celery workers, and PostgreSQL stored procedures to automate reporting tasks that previously required manual CSV preparation and repeated business-side validation.
- Implemented new shipment tracking integrations with **Express 4.x**, webhook verification, retry-safe HTTP clients, and audit logging, reducing missing tracking updates by **42%**.
- Resolved API timeout issues caused by N+1 ORM queries by introducing eager loading, pagination, Redis caching, and query-level profiling across Django and Node.js endpoints.
- Upgraded containerized services to Docker-based development environments, making local onboarding more reliable and reducing environment mismatch bugs between developer machines and staging servers.
- Designed background job workflows with Celery, Redis, cron jobs, and Python service modules to handle notifications, billing reminders, report generation, and third-party synchronization safely.
- Strengthened API security by adding JWT validation, request throttling, parameter sanitization, password hashing, and permission checks across older Django and Express applications.
- Collaborated with frontend, QA, DevOps, and client stakeholders to convert vague workflow requirements into maintainable backend APIs, database schemas, and production-ready release plans.

Webiz Digital — Software Developer *(Jan 2014 – Sep 2016)*

- Built digital agency, eCommerce, and marketing automation platforms using **Node.js 0.12.x–6.x**, **Python 2.7–3.5**, **Django 1.7.x–1.10.x**, Flask 0.10.x–0.11.x, Express 4.x, MySQL, MongoDB, jQuery, and Bootstrap.
- Developed backend APIs for product catalogs, campaign landing pages, customer forms, payment handoffs, and admin dashboards, helping small businesses launch production sites faster.
- Implemented Node.js middleware for form validation, email notifications, file uploads, and session handling, reducing repeated backend code across client projects by **29%**.

- Fixed a checkout issue where duplicated browser submissions created repeated orders by adding server-side token validation, transaction checks, and safer MySQL insert logic.
- Migrated older PHP and static-page workflows into **Django 1.8.x–1.10.x** applications with reusable templates, admin panels, authentication, and structured database models.
- Created Python scripts for CSV imports, customer cleanup, image processing, and campaign exports, saving account managers hours of manual spreadsheet work each week.
- Improved MongoDB-backed content features by adding indexes, query filters, and defensive schema validation, reducing slow admin search cases by **36%** on larger client datasets.
- Integrated third-party APIs for payment providers, email marketing tools, shipping services, and analytics platforms using Express routes, Python requests, and retry-aware error handling.
- Resolved production bugs in legacy JavaScript and Python code by tracing browser errors, server logs, malformed payloads, and database inconsistencies across mixed client environments.
- Supported deployment work with Git, Linux servers, Nginx, PM2, virtualenv, and basic shell automation, gaining flexible full-stack ownership across backend and frontend delivery.

Aurio — Junior Software Developer *(Oct 2012 – Dec 2013)*

- Supported internal business applications using **Python 2.7**, **Django 1.4.x–1.6.x**, early **Node.js 0.8.x–0.10.x**, JavaScript, jQuery, MySQL, HTML, CSS, Linux, and Git.
- Built small Django modules for user management, internal forms, report views, and admin screens while learning production coding standards from senior developers.
- Created Python automation scripts for log parsing, CSV cleanup, scheduled exports, and database maintenance tasks that reduced repetitive operational work for support teams.
- Fixed common backend bugs involving null values, broken form validation, incorrect date handling, and inconsistent database records by adding safer checks and clearer error messages.
- Assisted with early Node.js services for lightweight internal tools, using callback-based JavaScript, Express routing, JSON endpoints, and basic server-side validation.
- Improved MySQL query performance by adding indexes, simplifying joins, and reviewing slow query logs with senior engineers, reducing several internal report delays by **25%**.
- Helped migrate manual deployment notes into repeatable Linux shell scripts, Git-based release steps, and documented rollback procedures for safer small-team releases.
- Wrote unit tests and manual test cases for Django views, Python utility functions, and JavaScript form behavior to catch regressions before client review.
- Collaborated closely with product owners and QA to clarify requirements, reproduce issues, document fixes, and build confidence with **Python**, JavaScript, and database-driven applications.

Microsoft — Software Developer Intern *(Jun 2012 – Sep 2012)*

- Supported internal engineering tools using **C#**, **.NET Framework 4.0–4.5**, **ASP.NET MVC 3–4**, JavaScript, jQuery, and SQL Server, gaining practical experience in enterprise software quality and structured development workflows.
- Assisted senior engineers with debugging UI defects, validation issues, and data-handling problems, documenting root causes clearly so fixes could be reviewed and released with lower regression risk.
- Built small internal web modules for tracking records, reviewing status changes, and improving team visibility, applying clean HTML, CSS, JavaScript, and backend integration patterns.
- Resolved early-stage bugs related to inconsistent form validation and SQL query results by reproducing issues locally, checking logs, and working with mentors to apply safer input-handling rules.

Education

Texas State University
Bachelor's Degree in Computer Science (*Sep 2008 – May 2012)*